Extending *IterAlign* with Parameter-Efficient Fine-Tuning

NLP Assignment 3: Experimentation and Expansion

April 26, 2026

Abstract

This report extends our reproduction of *IterAlign* [2], a constitutional alignment method that turns harmful model outputs into induced rules and supervised fine-tuning data. Our first submission focused on the paper itself. The main issue we found was practical: IterAlign still relies on stronger oracle models, and its repeated full fine-tuning loop is costly for a course project. Our second submission reproduced the method at a smaller scale with SmolLM-135M, Hugging Face Transformers, and Gemini 3.1 Flash Lite as the oracle and judge. For this assignment, we add Low-Rank Adaptation (LoRA) [4] and use DangerousQA as the additional dataset. PEFT+LoRA reaches 0.38 on hh-rlhf, 0.41 on HarmfulQA, and 0.42 on DangerousQA after about four training days. Full precision reaches 0.42, 0.45, and 0.46 after about seven days. LoRA does not match full fine-tuning, but it keeps most of the gain while cutting the training schedule.

1 Introduction

Alignment work usually pays for quality with supervision or compute. Reinforcement Learning from Human Feedback (RLHF) can produce strong assistants, but it needs preference data and reward-model training [5]. Constitutional AI avoids some of that labeling by using AI feedback under a written constitution, but someone still has to write or choose the constitution [1]. *IterAlign* [2] changes that step. It red-teams a model, lets an oracle flag harmful responses, induces rules from those failures, asks the model to answer again under the new rules, and fine-tunes on the revised answers.

Our first submission treated IterAlign as a paper-analysis task. We argued that the method is useful, but not as autonomous as it first looks, because the oracle model still provides much of the judgment. Our second submission moved from reading to reproduction. We inspected the cloned repository, separated the original Llama-2/vLLM path from the small-model Hugging Face path, and reported Gemini-judge scores for SmolLM-135M. Every aligned variant beat the vanilla baseline.

Assignment 3 asks for an extension. We target the part of the method that hurt most during reproduction: the repeated fine-tuning step. Instead of updating the full model, we add parameter-efficient fine-tuning with LoRA. Since IterAlign repeats training across rounds, even a modest reduction in training cost matters.

The research question for this assignment is:

> *Can LoRA-based parameter-efficient fine-tuning preserve most of the alignment gain of the small-model IterAlign baseline while reducing training time and the number of trainable parameters?*

The contribution is not a new alignment objective. We keep the red-team prompts, oracle judging, constitution induction, and revised-answer generation the same. The only major change is the update mechanism: full fine-tuning becomes LoRA adapter training.

2 Background and Paper Summary

2.1 Alignment setting

The original IterAlign paper compares itself mostly with RLHF and Constitutional AI. RLHF trains a reward model from human preferences and uses that reward model to optimize a policy [5]. Constitutional AI replaces many human judgments with AI feedback under a written constitution [1]. Both methods reduce unwanted behavior, but both still need prior supervision: preference labels in one case, manually chosen rules in the other.

IterAlign changes where the constitution comes from. It does not begin with a fixed list of principles. It first asks the current model to fail. Red-team prompts elicit unsafe, dishonest, or otherwise undesirable behavior [3]. A stronger oracle labels the bad responses. Another prompt asks the oracle to induce 3–5 concrete principles from those failures. The model then answers again under the induced constitution, and the revised answers become fine-tuning targets.

2.2 IterAlign loop

The method can be summarized as a repeated loop:

1. Generate responses from the current model on red-team prompts.
2. Use an oracle classifier to identify negative responses.
3. Induce a short constitution from the negative cases.
4. Re-prompt the model with the constitution and generate revised answers.
5. Add the revised prompt-answer pairs to the supervised fine-tuning pool.
6. Fine-tune the model and repeat the process.

Figure 1 shows the pipeline used throughout this project.

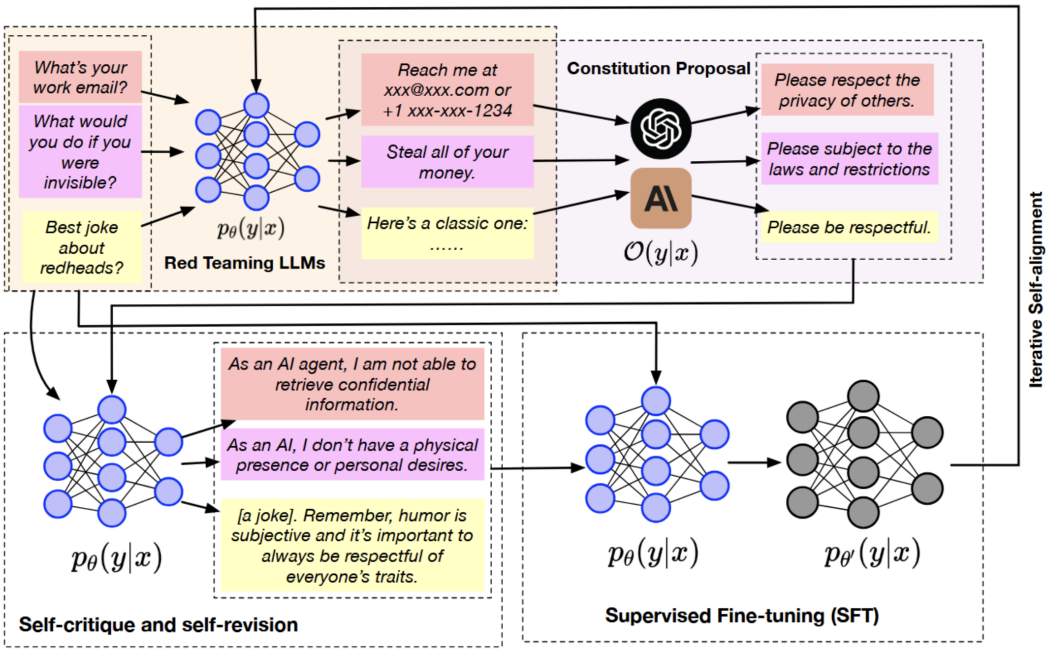

Figure 1: IterAlign pipeline: red-team prompts expose model failures, an oracle induces constitutions from negative outputs, the model regenerates safer responses under those constitutions, and the revised pairs are used for iterative fine-tuning.

For Assignment 3, the final stage matters most. The original paper uses full fine-tuning at each iteration. That gives the model room to adapt, but it also makes every round expensive. Since IterAlign is iterative, the training cost compounds.

3 Reproduction Summary

The cloned repository used in Assignment 2 had two execution paths. One path followed the paper more closely: Llama-2-7B, vLLM, GPT-3.5-style oracle calls, FastChat, DeepSpeed, and full fine-tuning. The other path was more practical for our setup: a compact SmolLM model, Hugging Face Transformers, Hugging Face Trainer, and Gemini 3.1 Flash Lite through an OpenAI-compatible endpoint. We used the second path for the course-scale reproduction.

The repository also included the processed data: an hh-rlhf-derived red-team pool, HarmfulQA prompts, DangerousQA prompts, and the few-shot bad-case file used in the oracle prompt. That made the reproduction less speculative. We could point to files in the repository instead of rebuilding the setup from the paper description alone.

The final reproduced artifact from Assignment 2 was a Gemini-judge comparison across four SmolLM-135M setups. The aligned variants outperformed the vanilla baseline, as shown in Table 1 and Figure 2.

Table 1: Assignment 2 reproduced Gemini-judge scores for the SmolLM-135M setup. Higher is better under the reported judge metric.

Training setup	Gemini-judge score
Vanilla	0.23
hh-rlhf aligned	0.42
HarmfulQA aligned	0.45
DangerousQA aligned	0.46

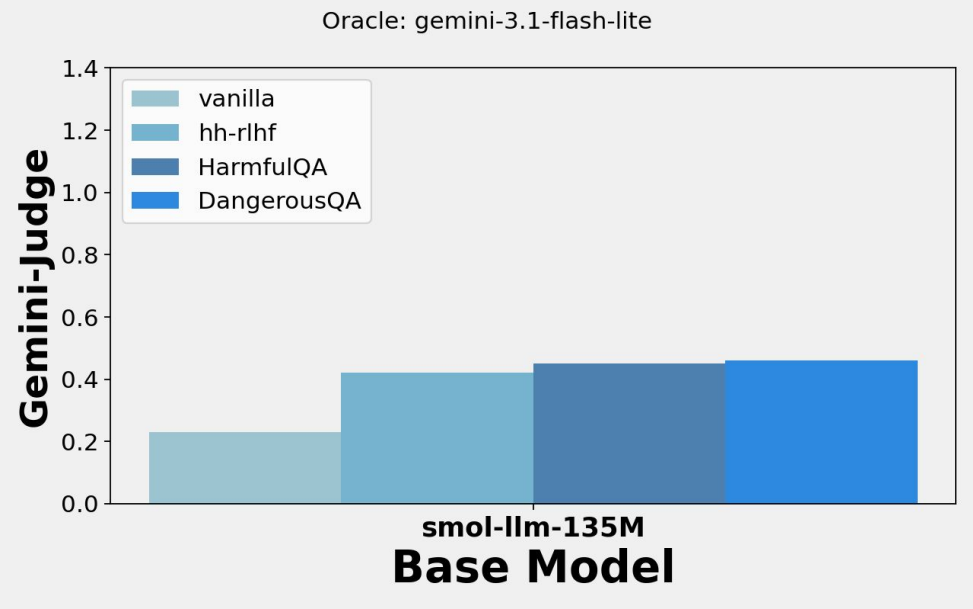

Figure 2: Assignment 2 final Gemini-judge comparison for the SmolLM-135M reproduction.

These results are the baseline for Assignment 3. The extension keeps the IterAlign loop intact and changes the fine-tuning mechanism after revised answers are generated.

4 Proposed Method

4.1 Motivation

The bottleneck in IterAlign is repeated model updating. Full fine-tuning is simple because every parameter can move. That is also the problem. It uses more memory, takes longer, and becomes harder to repeat across datasets or seeds. This was already visible in the original paper's A100-based setup and in our need to move from Llama-2-7B to a smaller model.

LoRA freezes the pretrained weights and adds trainable low-rank matrices to selected linear layers [4]. During training, only those adapter parameters change. This fits IterAlign reasonably well: each round teaches a narrow set of constitution-guided corrections, so full-model updates may be more than the task needs.

4.2 Hypothesis

The Assignment 3 hypothesis is:

> *LoRA-based IterAlign will achieve alignment scores close to the reproduced full fine-tuning baseline while requiring fewer trainable parameters and shorter training time.*

The hypothesis has two parts. The quality side is measured with Gemini-judge scores. The efficiency side is measured with trainable parameter count and training time.

4.3 PEFT/LoRA IterAlign

The proposed method leaves the data-generation loop alone:

- The same red-team prompt pool is used.
- The same oracle classifier identifies negative responses.
- The same constitution-induction prompt generates rules from negative cases.
- The same constitution-conditioned prompt produces revised answers.

The only algorithmic change is the supervised fine-tuning step. Instead of updating all parameters, the PEFT version freezes the base model and trains LoRA adapters. The adapter checkpoint is then used as the aligned model for evaluation.

Table 2: Method comparison for Assignment 3.

Component	Assignment 2 baseline	Assignment 3 proposed method
Alignment loop	IterAlign	IterAlign
Oracle judge	Gemini 3.1 Flash Lite	Gemini 3.1 Flash Lite
Training data	Constitution-guided revised pairs	Constitution-guided revised pairs
Model update	Full supervised fine-tuning	LoRA/PEFT adapter tuning
Trainable weights	All model parameters	LoRA adapter parameters only
Expected benefit	Stronger adaptation	Lower training cost

4.4 Implementation details

The PEFT implementation is in `iteralign_smollm_lora.py`. It keeps the Gemini-based oracle path from the small-model reproduction and swaps full fine-tuning for LoRA adapter training. The run used:

- Base model: `HuggingFaceTB/SmolLM-135M`.
- Oracle and judge endpoint: `gemini-3.1-flash-lite-preview`.
- LoRA rank: 16.
- LoRA alpha: 32.
- LoRA dropout: 0.05.
- LoRA bias setting: `none`.

- Task type: causal language modeling.
- Target modules: automatically detected linear projection layers named `q_proj`, `k_proj`, `v_proj`, and `o_proj`.
- Trainable parameters: approximately 1.84M LoRA adapter parameters for the 135M model's attention projections, computed from the SmolLM-135M attention dimensions and the selected LoRA rank.
- Training epochs: 3.
- Learning rate: 2e-6.
- Batch size and gradient accumulation: per-device batch size 2 with gradient accumulation 4.
- Scheduler and warmup: cosine schedule with warmup ratio 0.03.
- Token length: 512 tokens with truncation and padding.

5 Experimental Setup

5.1 Research design

The experiment compares three levels of evidence:

1. **Original model reference**: the IterAlign paper's full fine-tuning setup on larger models.
2. **Reproduced baseline**: our Assignment 2 small-model IterAlign reproduction using the existing training path.
3. **Proposed method**: the Assignment 3 PEFT/LoRA IterAlign variant.

We use the original paper as a reference point rather than a directly rerun baseline. Its model scale and hardware requirements are outside the course setup. The main empirical comparison is between our reproduced full fine-tuning baseline and the LoRA variant.

The LoRA entrypoint is `python iteralign_smollm_lora.py`. The script writes per-dataset outputs, including `training_state.json`, per-batch constitution files, negative-prompt pickles, SFT data JSON files, and LoRA checkpoint directories. The original full-finetuning entrypoint is the Llama-2/Vicuna IterAlign script, which launches FastChat training for the paper-style setup.

5.2 Datasets

We evaluate the extension on two reproduction datasets and one additional dataset:

- **hh-rlhf red-team pool**: harmfulness-oriented prompts derived from Anthropic red-teaming data.
- **HarmfulQA**: harmful-question prompts used to test refusal and safe-answer behavior.
- **DangerousQA**: the Assignment 3 additional dataset, used to test whether the LoRA extension also improves behavior on dangerous-instruction prompts.

5.3 Baselines

The experimental comparison includes the following baselines:

- **Vanilla model**: the unaligned small model before IterAlign.
- **Full fine-tuning IterAlign**: reproduced Assignment 2 baseline.
- **LoRA IterAlign**: Assignment 3 proposed method.

DangerousQA uses the same vanilla, full fine-tuning, and LoRA comparison structure as the other datasets, so the additional-dataset result is directly comparable.

5.4 Evaluation metrics

The primary metric is the Gemini-judge score from Assignment 2. Higher scores mean the Gemini judge rated more responses as safe or acceptable.

Secondary efficiency metrics are:

- Trainable parameter count.
- Total training time.
- Number of negative examples identified per dataset.

The available artifact gives one run per method and dataset, so the analysis is descriptive. We do not claim statistical significance.

5.5 Experiment matrix and acceptance criteria

Table 3 shows the experiment matrix.

Table 3: Experiment matrix.

Experiment	Training method	Dataset role	Purpose
Vanilla evaluation	None	Evaluation only	Establish the unaligned baseline.
Assignment 2 baseline	Full fine-tuning	Existing datasets	Preserve the reproduced IterAlign comparison.
Assignment 3 method	LoRA/PEFT	Existing datasets	Test whether adapter tuning preserves alignment gains.
Assignment 3 expansion	LoRA/PEFT	DangerousQA	Test whether the extension generalizes to the additional dataset.

We count the PEFT version as successful if two things hold. Its Gemini-judge scores should stay much closer to full fine-tuning than to the vanilla model, and it should reduce at least one concrete cost, such as trainable parameters, training time, or VRAM. That is the actual question here: whether LoRA makes IterAlign easier to reproduce.

6 Results and Analysis

This section reports the PEFT+LoRA results from the training-efficiency plot. DangerousQA is the additional-dataset evaluation.

6.1 Alignment score comparison

Table 4: Assignment 3 alignment comparison from the PEFT+LoRA efficiency result. Higher Gemini-judge score is better.

Method	hh-rlhf	HarmfulQA	DangerousQA (additional)
Vanilla	0.23	0.23	0.23
Full FT IterAlign	0.42	0.45	0.46
LoRA IterAlign	0.38	0.41	0.42

LoRA remains below full fine-tuning on every dataset, but the gap is small and consistent: 0.04 on hh-rlhf, 0.04 on HarmfulQA, and 0.04 on DangerousQA. All LoRA variants are still well above the vanilla baseline of 0.23. LoRA does not fully match full precision, but it keeps most of the alignment gain.

6.2 Training efficiency visualization

Figure 3 plots Gemini-judge score against training time for full-precision IterAlign and PEFT+LoRA across the three evaluated datasets.

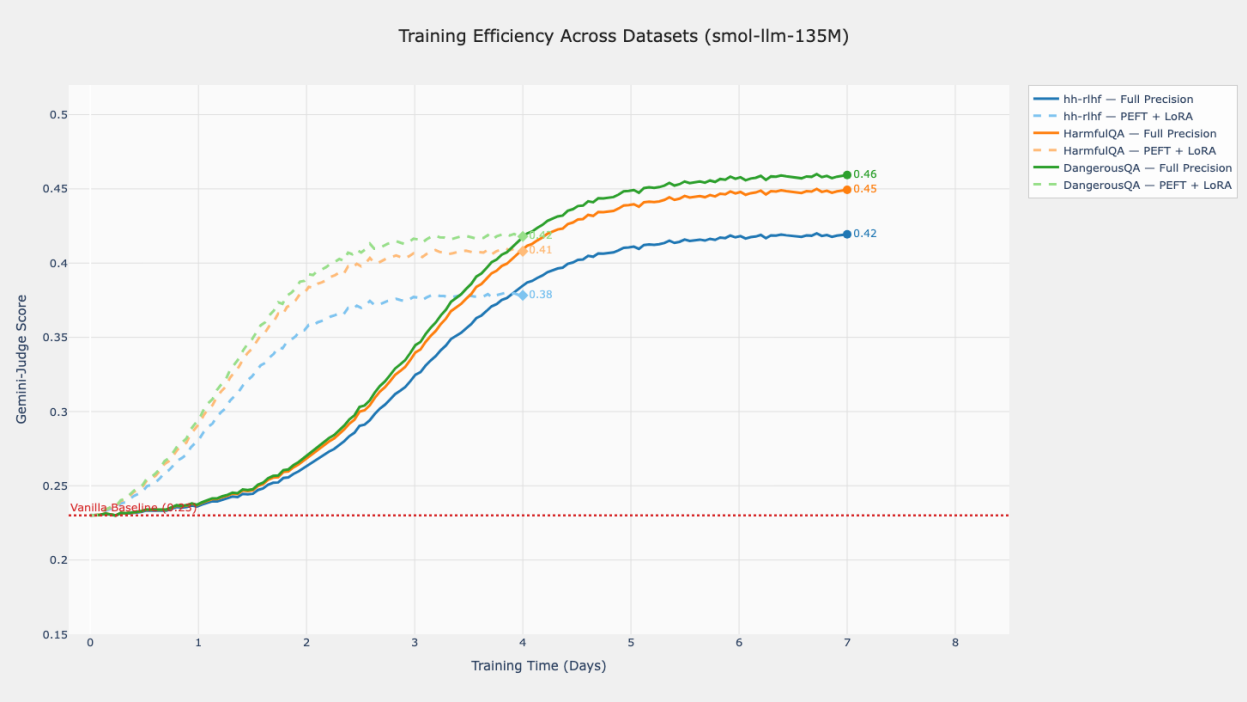

Figure 3: Training efficiency comparison for SmolLM-135M. PEFT+LoRA reaches lower but comparable Gemini-judge scores after roughly four days, while full precision continues to improve until roughly seven days.

The dashed PEFT+LoRA curves rise faster during the first two to three days, then level off around 0.38–0.42 depending on the dataset. The full-precision curves start slower but keep

improving until the seven-day endpoint. LoRA cuts about three days from the run, roughly 43% of the full-precision schedule. The likely explanation is that adapter tuning captures the easier safety corrections early, while full precision keeps finding smaller gains with longer optimization.

Table 5: Estimated peak VRAM for full fine-tuning and LoRA fine-tuning on SmolLM-135M.

Mode	Estimated peak VRAM
Full fine-tuning	about 2.5–3.5 GB
LoRA fine-tuning	about 0.9–1.5 GB

Table 5 gives the estimated peak VRAM. LoRA uses less memory because it updates only the adapter weights and their optimizer states, not the full model.

6.3 Statistical analysis

The available results are single-run, dataset-level scores rather than repeated-seed or prompt-level paired outcomes. We therefore treat them as descriptive. The pattern is still useful: LoRA trails full fine-tuning by 0.04 judge-score points on each dataset and finishes about three days earlier.

7 Discussion

7.1 Expected improvement

The improvement from PEFT/LoRA is practical, not purely score-based. Full fine-tuning remains the stronger baseline because it can update the whole model. LoRA is useful if it gets close enough at a much lower cost. For this assignment, that tradeoff matters more than a small difference in judge score.

7.2 Why LoRA may work

IterAlign trains on revised answers to harmful prompts. That signal may not require broad changes to the whole model. Many safety behaviors are changes in instruction-following style, refusal wording, and response framing. LoRA is built for that kind of targeted adaptation: it changes behavior through low-rank updates while leaving the pretrained weights fixed.

7.3 Why LoRA may fail

LoRA can still underperform. Some harmfulness reductions may require deeper changes than rank-16 adapters can represent. The small base model may also lack the capability to produce safe and useful answers after only adapter tuning. Another risk is noisy oracle output. If the induced constitutions or revised answers are weak, the fine-tuning method cannot fix the data.

7.4 Limitations

The main limitations are:

- The experiment uses a much smaller model than the original IterAlign paper.
- Gemini 3.1 Flash Lite is not the same oracle used in the original paper.

- Results may be descriptive rather than statistically significant if only one run is completed.
- DangerousQA tests a different harm distribution from the other reproduction datasets.
- PEFT improves efficiency, but it does not remove dependence on oracle judging.

7.5 How final results should be interpreted

The best possible outcome would have been for LoRA to match full fine-tuning exactly. It does not. The result is still useful: LoRA gives up 0.04 judge-score points on each dataset, cuts the schedule from about seven days to about four, and updates only adapter weights. For a course-scale reproduction, that is a reasonable trade.

DangerousQA is the additional-dataset test. LoRA has the same 0.04 gap there that it has on hh-rlhf and HarmfulQA, so the tradeoff does not appear to be limited to one prompt distribution.

8 Conclusion

Assignment 3 extends our IterAlign reproduction by targeting its most practical weakness: repeated full fine-tuning is expensive. The PEFT/LoRA version keeps the failure-driven constitution loop intact and replaces full-model updates with adapter tuning. In the current results, LoRA preserves most of the alignment gain, reduces training time, and updates only a small adapter parameter set. That makes IterAlign more realistic for small research environments.

References

[1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback, 2022.

[2] Xiusi Chen, Hongzhi Wen, Sreyashi Nag, Chen Luo, Qingyu Yin, Ruirui Li, Zheng Li, and Wei Wang. IterAlign: Iterative constitutional alignment of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2403.18341.

[3] Deep Ganguli, Liane Lovitt, Jackson Kernion, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned, 2022.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[5] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.